

JSON Path in K8s

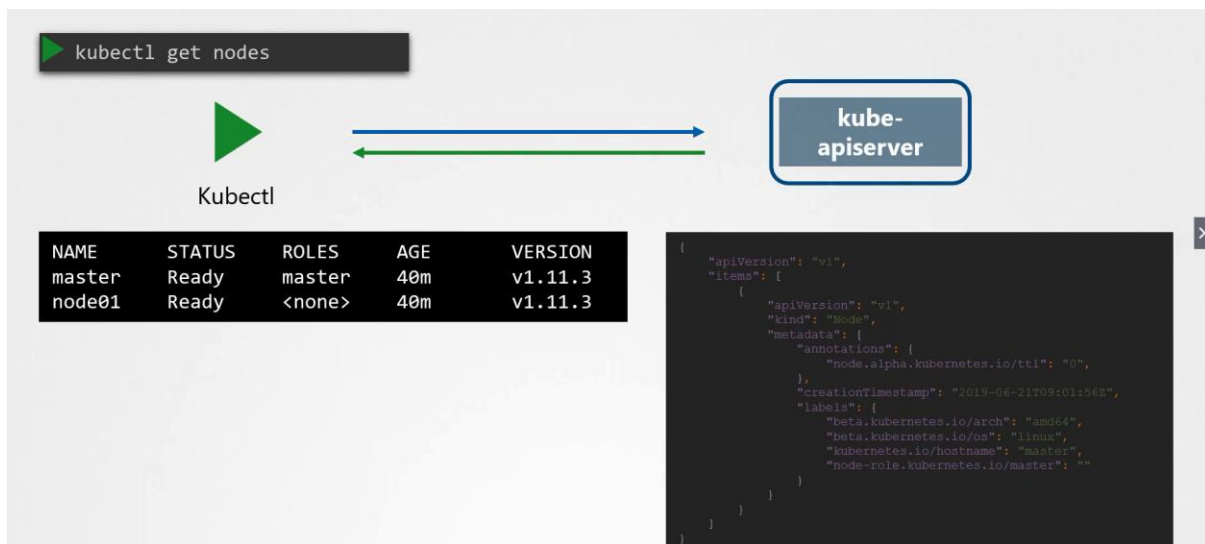
Why JSON Path?

UC: Large data sets

- 100's of nodes
- 1000's of Pods, Deployments, ReplicaSet

What is Kubectl?

Is a K8s CLI Utility, used to read and write K8s objects? Every time you run a kubectl command it interacts with kube-apiserver. Kube-apiserver knows the json language so it returns the requested information in json format. kubectl receives the json response and convert it in the human readable format and prints in the screen, during that process lots of the json information is hidden in order to make them human readable.



If you want to see the additional information, use the **-o wide** option.

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	40m	v1.11.3
node01	Ready	<none>	40m	v1.11.3

```
{
  "apiVersion": "v1",
  "items": [
    {
      "apiVersion": "v1",
      "kind": "Node",
      "metadata": {
        "annotations": {
          "node.alpha.kubernetes.io/ttl1": "0",
        },
        "creationTimestamp": "2019-06-21T09:01:56Z",
        "labels": {
          "beta.kubernetes.io/arch": "amd64",
          "beta.kubernetes.io/os": "linux",
          "kubernetes.io/hostname": "master",
          "node-role.kubernetes.io/master": ""
        }
      }
    }
  ]
}
```

```
➤ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE
master	Ready	master	7m	v1.11.3	172.17.0.44	<none>	Ubuntu 16.04.2 LTS
node01	Ready	<none>	6m	v1.11.3	172.17.0.63	<none>	Ubuntu 16.04.2 LTS

But still this is not complete there is lots of information is still hidden and not shown in the output e.g., resource capacity available on this node, taints sets on the node etc.

How to JSON Path in kubectl?

How to JSON PATH in KubeCtl?

- 1 Identify the **kubectl** command
- 2 Familiarize with **JSON** output
- 3 Form the **JSON PATH** query

'\$' not mandatory. Kubectl adds it

```
.items[0].spec.containers[0].image
```
- 4 Use the **JSON PATH** query with **kubectl** command


```
kubectl get pods -o=jsonpath='{ .items[0].spec.containers[0].image }'
```

```
{
  "apiVersion": "v1",
  "kind": "List",
  "items": [
    {
      "apiVersion": "v1",
      "kind": "Pod",
      "metadata": {
        "name": "nginx-5557945897-gznjp",
      },
      "spec": {
        "containers": [
          {
            "image": "nginx:alpine",
            "name": "nginx"
          }
        ],
        "nodeName": "node01"
      }
    }
  ]
}
```

How to JSON PATH in kubectl?

Identify the kubectl command? **kubectl get nodes**

Familiarize with JSON output (**-o json**) **kubectl get nodes -o json**

Form the JSON PATH Query

JSON PATH Query: With the help of json path query you can filter, and format the output of the command as you like.

```
.items[*].status
```

Use the JSON PATH Query with kubectl command

Note: encapsulate the jsonpath query with '{}'

```
kg nodes -o jsonpath='{.items[*]}'
```

```
kg node -o jsonpath='{.items[*].status.addresses[].address}'
```

JSON PATH – Wildcard

DATA	QUERY	RESULT
<pre>{ "car": { "color": "blue", "price": 20000, "wheels": [{ "model": "X345ERT", }, { "model": "X346ERT", }] }, "bus": { "color": "white", "price": 120000, "wheels": [{ "model": "Z227KLJ", }, { "model": "Z226KLJ", }] } }</pre>	Get car's 1 st wheel model \$.car.wheels[0].model	["X345ERT"]
	Get car's all wheel model \$.car.wheels[*].model	["X345ERT", "X346ERT"]
	Get bus's wheel models \$.bus.wheels[*].model	["Z227KLJ", "Z226KLJ"]
	Get all wheels' models \$.*.wheels[*].model	["X345ERT", "X346ERT", "Z227KLJ", "Z226KLJ"]

JSON PATH – Example

JSON PATH Examples

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name}'
```

```
master node01
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.nodeInfo.architecture}'
```

```
amd64 amd64
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.capacity.cpu}'
```

```
4 4
```

```
{
  "apiVersion": "v1",
  "items": [
    {
      "apiVersion": "v1",
      "kind": "Node",
      "metadata": {
        "name": "master"
      },
      "status": {
        "capacity": {
          "cpu": "4"
        },
        "nodeInfo": {
          "architecture": "amd64",
          "operatingSystem": "linux"
        }
      }
    },
    {
      "apiVersion": "v1",
      "kind": "Node",
      "metadata": {
        "name": "node01",
      },
      "status": {
        "capacity": {
          "cpu": "4",
        },
        "nodeInfo": {
          "architecture": "amd64",
          "operatingSystem": "linux",
        }
      }
    }
  ]
}
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name}'
```

```
master node01
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.nodeInfo.architecture}'
```

```
amd64 amd64
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.capacity.cpu}'
```

```
4 4
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name}{.items[*].status.capacity.cpu}'
```

```
master node01 4 4
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name}'
```

```
master node01
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.nodeInfo.architecture}'
```

```
amd64 amd64
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].status.capacity.cpu}'
```

```
4 4
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name}{.items[*].status.capacity.cpu}'
```

```
master node01 4 4
```

```
➤ kubectl get nodes -o=jsonpath='{.items[*].metadata.name} {"\n"} {.items[*].status.capacity.cpu}'
```

```
master node01
```

```
4 4
```

`{"\n"}` New line

`{"\t"}` Tab

Custom-Columns

kg nodes -o custom-columns=<COLUMN_NAME>:<JSONPATH> # not a querypath

kg nodes -o custom-columns=NODE:.metadata.name,CPU:.status.capacity.cpu

kg pod -A -o=custom-columns=NAME:.metadata.name,CPU:.status.capacity.cpu --sort-by=.status.capacity.cpu

```
kubectl get nodes -o=jsonpath='{.items[*].metadata.name}{"\n"}{.items[*].status.capacity.cpu}'
```

NAME	CPU
master	4
node01	4

```
kubectl get nodes -o=custom-columns=<COLUMN NAME>:<JSON PATH>
```

```
kubectl get nodes -o=custom-columns=NODE:.metadata.name ,CPU:.status.capacity.cpu
```

NAME	CPU
master	4
node01	4

SORT-BY

kg nodes --sort-by=.metadata.name # not a querypath

kg nodes --sort-by=.status.capacity.cpu

```
kubectl get nodes -o=custom-columns=NODE:.metadata.name ,CPU:.status.capacity.cpu
```

NAME	CPU
master	4
node01	4

```
kubectl get nodes --sort-by=.metadata.name
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	5m	v1.11.3
node01	Ready	<none>	5m	v1.11.3

```
kubectl get nodes --sort-by=.status.capacity.cpu
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	5m	v1.11.3
node01	Ready	<none>	5m	v1.11.3